

2º Prompt Gemini

Guía Práctica de Code Review: Prevención de Inyecciones SQL (SQLi)

Un proceso de revisión de código (Code Review) enfocado en seguridad busca garantizar que ninguna entrada provista por un usuario sea interpretada como comandos SQL ejecutables por la base de datos.

1. Protocolo de Revisión Paso a Paso

Sigue esta secuencia cada vez que revises un Pull Request (PR) que modifique o añada interacción con bases de datos:

Paso 1: Ubicar los puntos de entrada de datos

Identifica dónde entran datos externos a la aplicación. Ejemplos habituales:

- Parámetros en la URL (p. ej., `/api/usuarios?id=123`)
-
- Datos del cuerpo de las peticiones HTTP (`JSON`, `form-data`)
-
- Encabezados (*Headers*) o Cookies
-
- Parámetros de búsqueda u ordenación (`sort`, `filter`)
-

Paso 2: Seguir el recorrido del dato (*Taint Analysis* básico)

Rastrea cómo viaja esa variable desde la entrada (controlador/ruta) hasta la capa de persistencia (repositorio/DAO/base de datos).

Paso 3: Analizar la construcción de la consulta SQL

Inspecciona la línea exacta donde se ejecuta la consulta a la base de datos:

- ¿Se concatenan o formatean cadenas? (*Bandera roja*)
-
- ¿Se usan marcadores de posición (*placeholders*) o métodos seguros del ORM? (*Práctica recomendada*)
-

2. Detección de Vulnerabilidades

Durante la revisión, busca específicamente estos patrones de código vulnerables:

Patrón Vulnerable 1: Concatenación o formateo de cadenas

Si ves símbolos de suma (+), interpolación de cadenas (`f"..."`, ``${...}``) o funciones como `format()` al construir la consulta SQL, **el código es vulnerable**.

- Ejemplo vulnerable (Python):

-

Python

INCORRECTO: Si el usuario envía "1 OR 1=1", devolverá todos los registros

```
"SELECT * FROM usuarios WHERE email = ""
```

-

- Ejemplo vulnerable (Node.js/JavaScript):

-

JavaScript

// INCORRECTO: Interpolación de texto directa

```
const `SELECT * FROM productos WHERE categoria = '${req.query.categoria}'`
```

```
await
```

-

Patrón Vulnerable 2: Consultas nativas (*Raw Queries*) dentro de un ORM

Aunque se use un ORM (Sequelize, TypeORM, SQLAlchemy, Hibernate, etc.), el uso de funciones de consulta nativa pasando texto directo reintegra el riesgo de SQLi.

- Ejemplo vulnerable (ORM con SQL directo):

-

Python

INCORRECTO: Uso de raw SQL con formateo de cadena

```
f"UPDATE usuarios SET estado = 'inactivo' WHERE id = {user_id}"
```

-

Patrón Vulnerable 3: Ordenación o nombres de tablas dinámicos

Los marcadores de posición estándar no suelen funcionar para nombres de columnas o palabras clave como **ASC/DESC**. Si el código permite que el usuario decida por qué columna ordenar, revisa que no se inserte el valor directamente.

- Ejemplo vulnerable:

-

Python

INCORRECTO: Permite inyección en el nombre de la columna

```
f"SELECT * FROM productos ORDER BY {columna_usuario} ASC"
```

-

3. Corrección de Vulnerabilidades

Si detectas alguno de los patrones anteriores durante el Code Review, solicita los siguientes cambios:

Solución 1: Utilizar Consultas Parametrizadas (Sentencias Preparadas)

La solución principal es separar la estructura de la consulta de los datos de entrada. Los datos se envían de forma independiente y el motor de base de datos garantiza que se traten estrictamente como texto o números, no como código ejecutable.

- Corrección (Python):
-

Python

```
# CORRECTO: Se utiliza el marcador de posición %s y se pasan los datos en una tupla
"SELECT * FROM usuarios WHERE email = %s"
```

-
- Corrección (Node.js/PostgreSQL):
-

JavaScript

```
// CORRECTO: Se utilizan marcadores numerados ($1, $2, etc.)
const "SELECT * FROM productos WHERE categoria = $1"
await
```

Solución 2: Aprovechar las funciones nativas del ORM

Utiliza la API propia del ORM en lugar de escribir SQL manual siempre que sea posible.

- Corrección (ORM):
-

Python

```
# CORRECTO: El ORM gestiona la parametrización automáticamente
```

Solución 3: Usar Listas Blancas (*Allowlists*) para identificadores

Para casos donde no se puedan usar parámetros (como en `ORDER BY` o nombres de tabla), valida la entrada contra una lista con valores permitidos predefinidos.

- Corrección:
-

Python

```
# CORRECTO: Validación estricta mediante lista blanca
```

```
"precio" "precio"
"nombre" "nombre"
"fecha" "fecha_creacion"
```

Si la entrada del usuario no está en la lista, se asigna un valor por defecto seguro

```
f"SELECT * FROM productos ORDER BY {columna_segura} ASC"
"fecha_creacion"
```

4. Lista de Comprobación (*Checklist*) Rápida para el Revisor

Al aprobar el Pull Request, confirma que el código cumple con los siguientes puntos:

- ☐ No existen concatenaciones de texto en ninguna instrucción SQL.
- ☐ Todas las variables externas se pasan como parámetros independientes a la consulta.
- ☐ Las consultas nativas en ORM se han sustituido por métodos de la API del ORM o se han parametrizado adecuadamente.
- ☐ Los nombres de columnas o tablas dinámicas se validan obligatoriamente con una lista blanca (*allowlist*).